Module No. 4	Network Layer
IP addressing schemes, IPV4, Subnetting, IPV6, shift from IPV4 to IPV6, ICMP, DHCP, ARP. Routing Protocols: Distance-vector and link-state routing. RIP, OSPF, BGP.	

Internet addressing scheme

- Allows users and applications to identify a specific network or host with which to communicate.
- TCP/IP provides standards for assigning addresses to networks, subnetworks, hosts, and sockets, and for using special addresses for broadcasts and local loopback.
- Internet addresses are made up of
    - ✓ A Network address
    - ✓ A Host (or local) address
- A unique, official network address is assigned to each network when it connects to other Internet networks.

- The Internet addressing scheme consists of
  - ✓ Internet Protocol (IP) addresses
  - ✓ Two special cases of IP addresses
    - ❖ Broadcast addresses
    - ❖ Loopback addresses

- **Internet addresses**

  The Internet Protocol (IP) uses a 32-bit, two-part address field.

- **Subnet addresses**

  Subnet addressing allows an autonomous system made up of multiple networks to share the same Internet address.

- **Broadcast addresses**

  The TCP/IP can send data to all hosts on a local network or to all hosts on all directly connected networks. Such transmissions are called broadcast messages.

- **Local loopback addresses**

  The Internet Protocol defines the special network address, **127.0.0.1**, as a local loopback address.

IPv4 Addressing

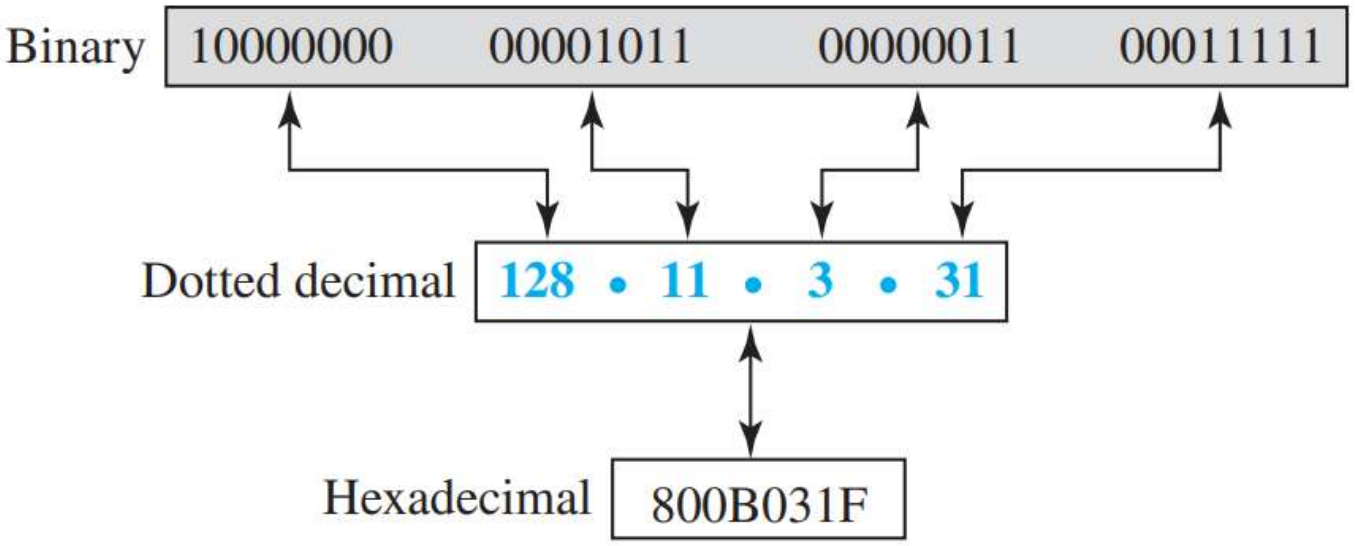
- The identifier used to identify the connection of each device to the Internet is called the Internet address or IP address.
- An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet.

Address Space

- It is the total number of addresses used by the protocol.
- If a protocol uses ***b bits*** to define an address, the address space is $\mathbf{2^b}$ because each bit can have two different values (0 or 1).
- IPv4 uses 32-bit addresses ➔ address space is $\mathbf{2^{32}}$ or 4,294,967,296

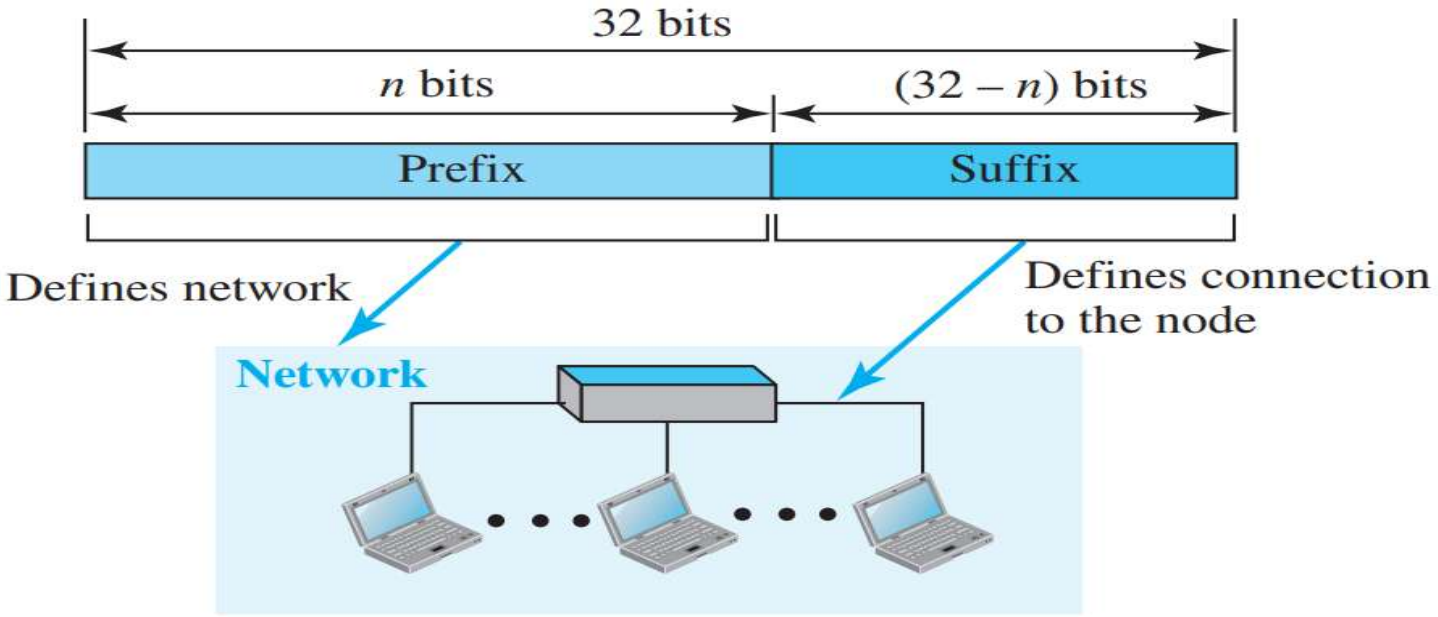
Notation

- Three common notations to show an IPv4 address:
  - ✓ Binary notation (base 2)
  - ✓ Dotted-decimal notation (base 256)
  - ✓ Hexadecimal notation (base 16)

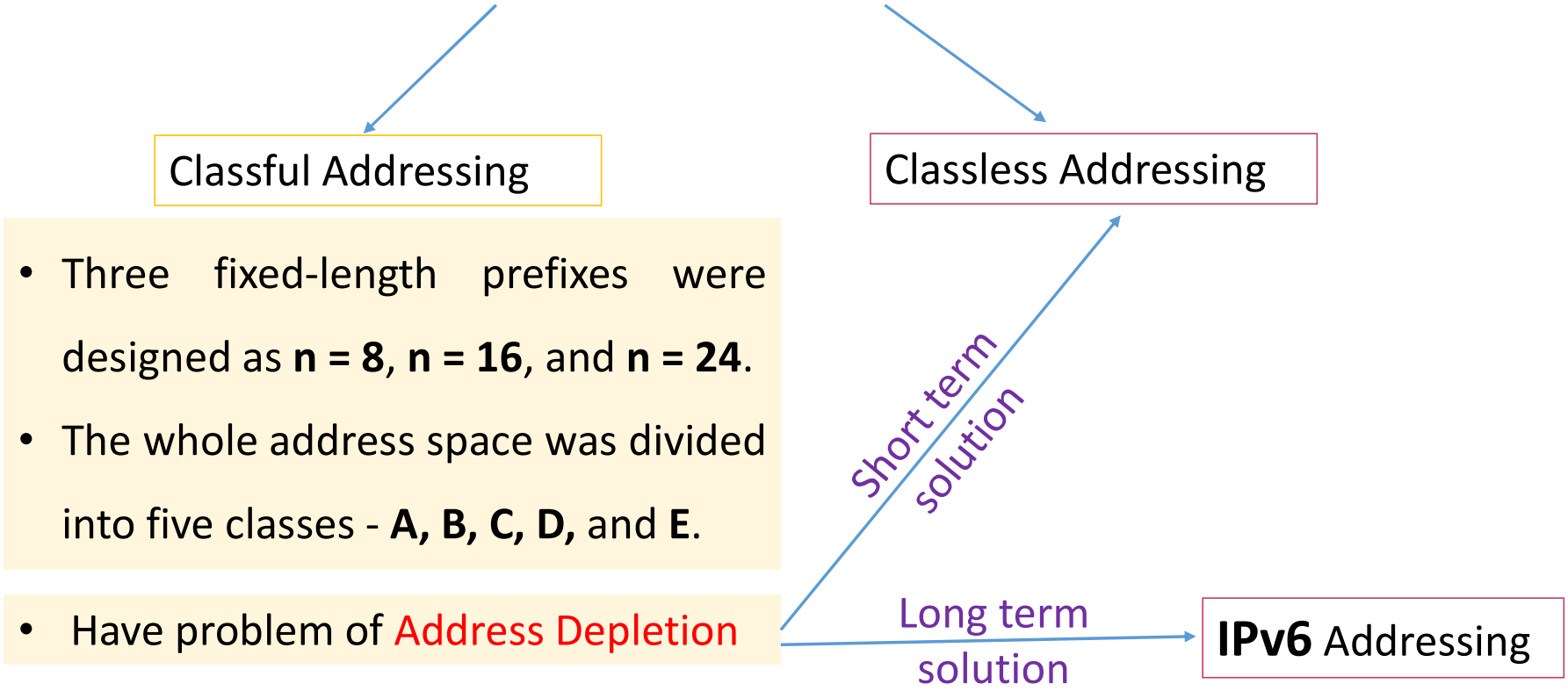


Hierarchy in Addressing

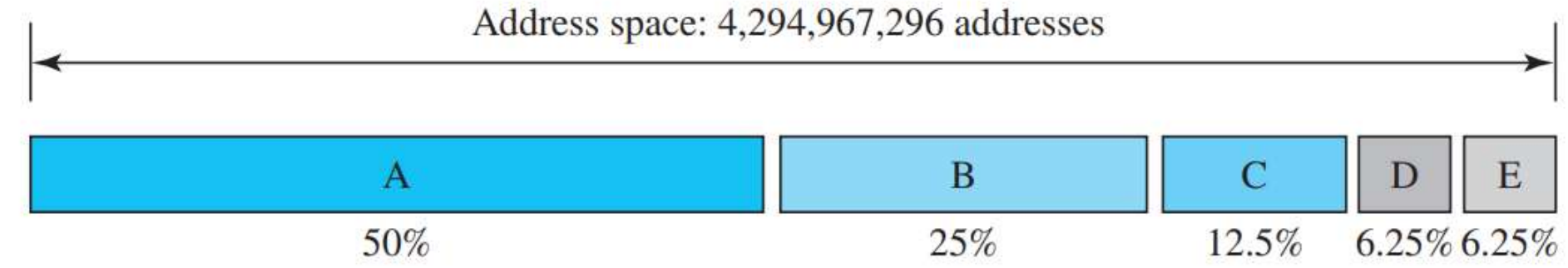
- This 32-bit IPv4 addressing system is hierarchical but is divided only into two parts.
  - ✓ **Prefix** - first part of the address, defines the ***network***
  - ✓ **Suffix** - second part of the address, defines the ***node***
- If the prefix length is **n *bits***, then the suffix length is **(32 – n) *bits***.

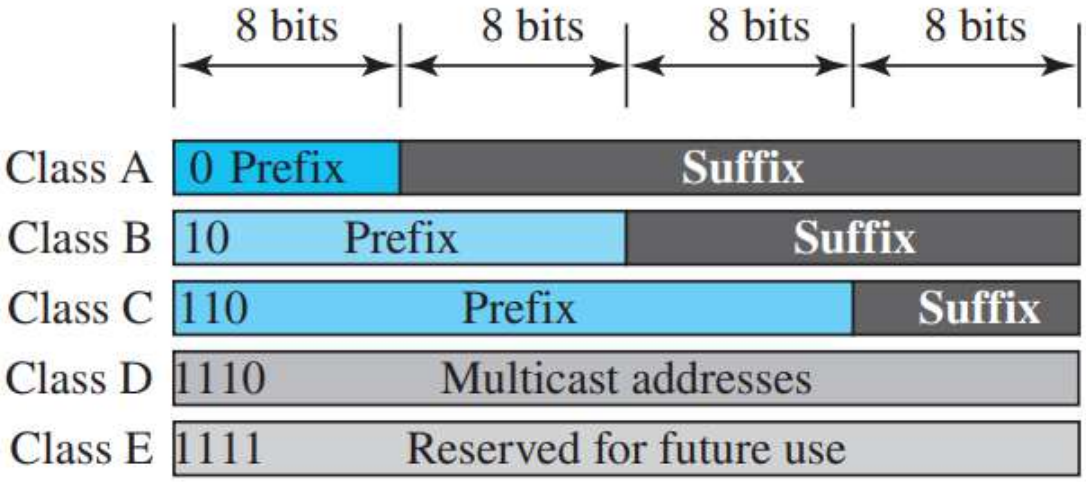


- A prefix can be of fixed length or variable length.



Classful Addressing





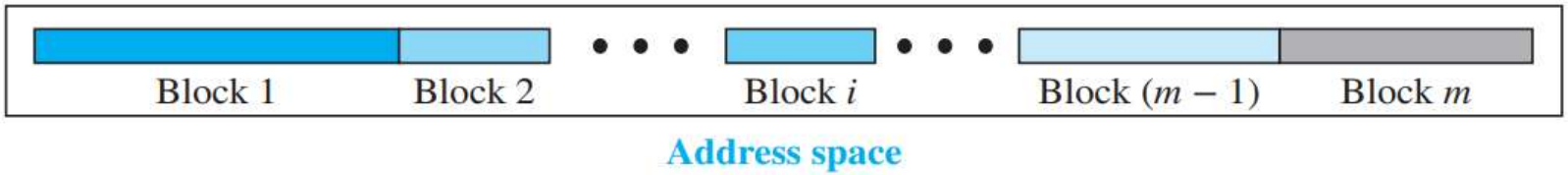
Class	Prefixes	First byte
A	$n = 8$ bits	0 to 127
B	$n = 16$ bits	128 to 191
C	$n = 24$ bits	192 to 223
D	Not applicable	224 to 239
E	Not applicable	240 to 255

Classless Addressing

- To overcome the ***Address Depletion*** problem of ***Classful addressing***, this was introduced.
- In this, the class privilege was removed from the distribution to compensate for the address depletion.

- Another reason to introduce Classless addressing is the advent of ISPs.
- In 1996, the Internet authorities announced a new architecture called ***Classless addressing***.

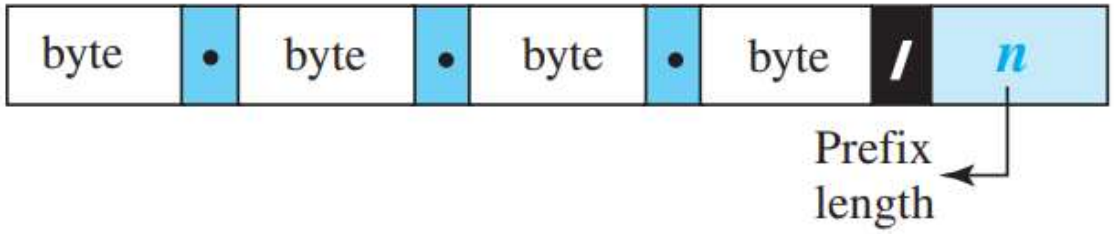
- In classless addressing, the whole address space is divided into variable-length blocks.
- One of the ***restriction***s is that the number of addresses in a ***block needs to be a power of 2***.
- An organization can be granted one block of addresses.



Address space

Prefix Length: Slash Notation

- Because the prefix length is not inherent in the address of ***Classless addressing***, it is needed to separately give the length of the prefix.
- In this case, the ***prefix length***, **n**, is added to the address, separated by ***a slash***.
- The notation is informally referred to as ***slash notation*** and formally as ***Classless InterDomain Routing (CIDR)*** strategy.



Examples:
12.24.76.8/8
23.14.67.92/12
220.8.24.255/25

Extracting Information from an Address

- Three pieces of information about the block to which the address belongs can be derived:
  - ✓ The number of addresses
  - ✓ The first address in the block
  - ✓ The last address
- If the value of prefix address length, **n**, is given, then
  - ✓ To find the first address, keep the n leftmost bits and set the (32 − n) rightmost bits all to 0s.
  - ✓ To find the last address, keep the n leftmost bits and set the (32 − n) rightmost bits all to 1s.

A classless address is given as 167.199.170.82/27. We can find the desired three pieces of information as follows. The number of addresses in the network is $2^{32-n} = 2^5 = 32$ addresses. The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

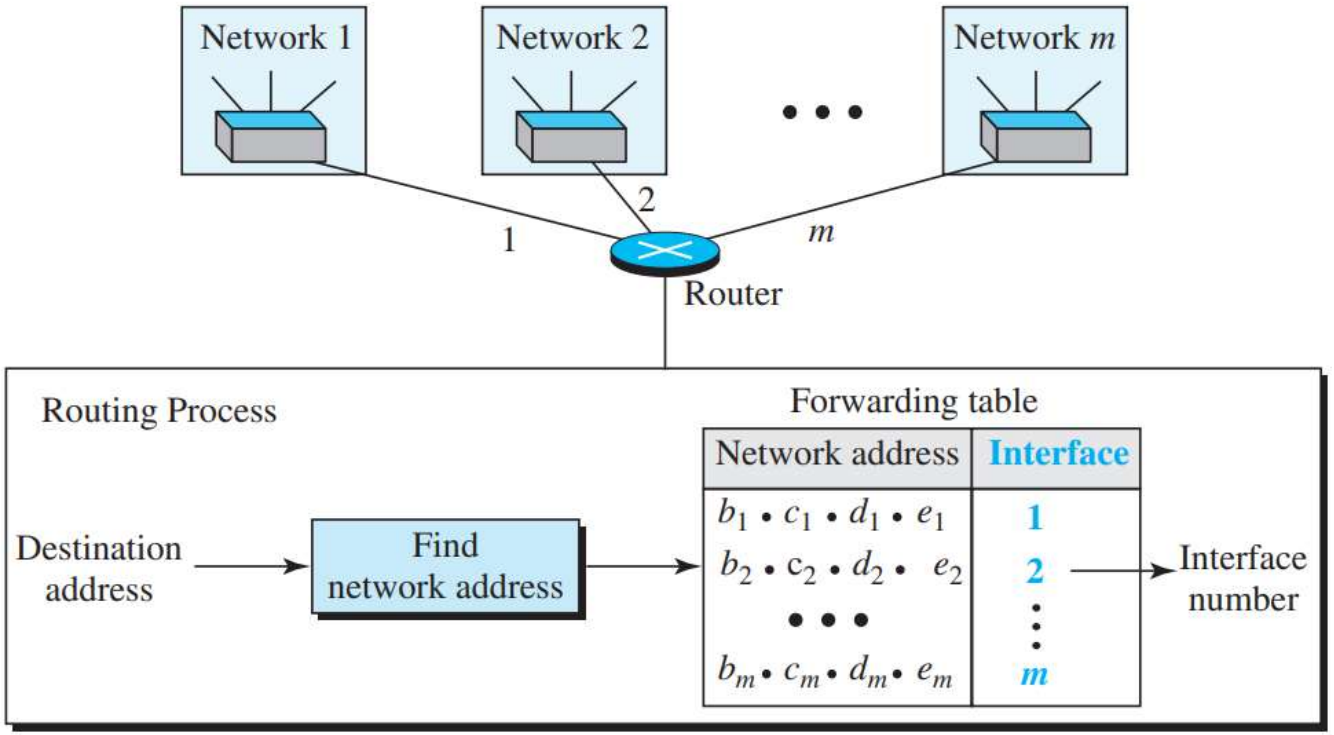
Address: 167.199.170.82/27	10100111	11000111	10101010	01010010
First address: 167.199.170.64/27	10100111	11000111	10101010	01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

Network Address

- The first address, the network address, is particularly important because it is used in routing a packet to its destination network.



For example,

1. An internet is made up of ***m networks*** and a router with ***m interfaces***.
2. When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent and from which interface the packet should be sent out.
3. After the network address has been found, the router consults its forwarding table to find the corresponding interface from which the packet should be sent out.
4. The network address is actually the identifier of the network; each network is identified by its network address.
5. When the packet arrives at the network, it reaches its destination host using link-layer addressing.

Block Allocation

- A global authority called the ***Internet Corporation for Assigned Names and Numbers (ICANN)***.
- It assigns a large block of addresses to an ISP (or a larger organization).
- For the proper operation of the CIDR, ***two restrictions*** need to be applied to the allocated block.
  1. The number of requested addresses, N, needs to be a power of 2.
     - The reason is that $N = 2^{32-n}$ or $n = 32 - \log_2 N$. If N is not a power of 2, cannot have an integer value for n.

2. The requested block needs to be allocated where there are a contiguous number of available addresses in the address space. There is a restriction on choosing the first address in the block.
   - The first address needs to be divisible by the number of addresses in the block.
   - The reason is that the first address needs to be the prefix followed by (32 − n) number of 0s.
   - The decimal value of the first address is then

$$\text{First address} = (\text{prefix in decimal}) \times 2^{32-n} = (\text{prefix in decimal}) \times N$$

Example

An ISP has requested a block of 1000 addresses. Because 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/22, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Subnetting

- More levels of hierarchy can be created using ***Subnetting***.
- An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a ***Subnetwork*** or ***Subnet***.
- Note that a subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

Designing Subnets

- The subnetworks in a network should be carefully designed to enable the routing of packets.
- If the total number of addresses granted to the organization is **N**, the prefix length is **n**,
  - ✓ the assigned number of addresses to each subnetwork is $\mathbf{N_{sub}}$,
  - ✓ the prefix length for each subnetwork is $\mathbf{n_{sub}}$.

The following steps need to be carefully followed to guarantee the proper operation of the subnetworks.

- ✓ The number of addresses in each subnetwork should be a **power of 2**.
- ✓ The prefix length for each subnetwork should be found using the following formula:

$$n_{sub} = 32 - \log_2 N_{sub}$$

- ✓ The starting address in each subnetwork should be divisible by the number of addresses in that subnetwork. This can be achieved if we first assign addresses to larger subnetworks.

Example

An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have three subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Solution

There are $2^{32-24} = 256$ addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

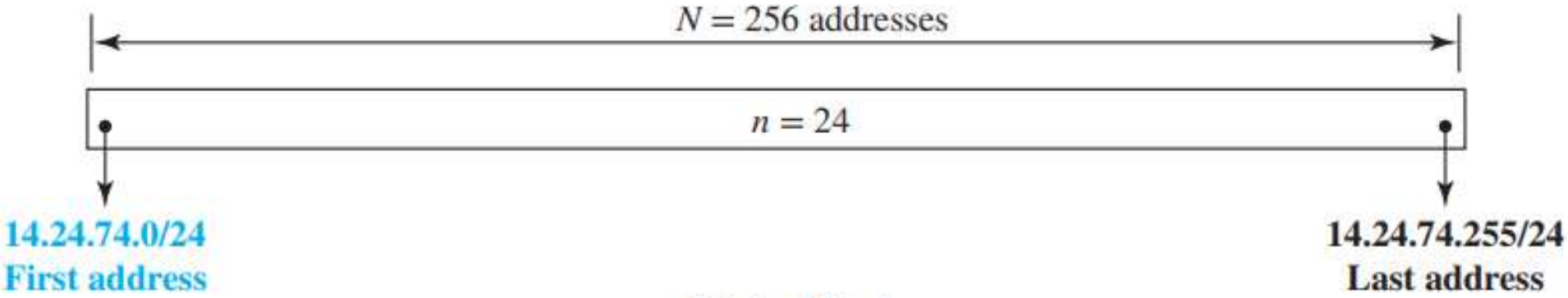
a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as

$n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/25; the last address is 14.24.74.127/25.

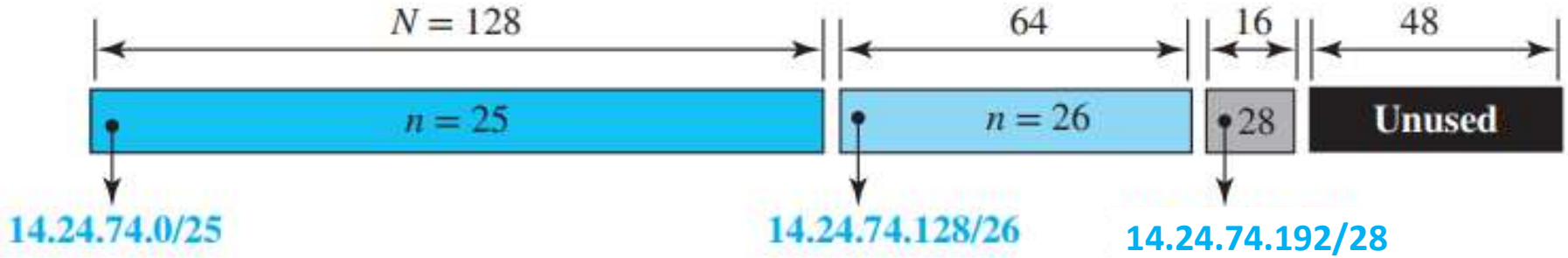
b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as $n_2 = 32 - \log_2 64 = 26$. The first address in this block is 14.24.74.128/26; the last address is 14.24.74.191/26.

c. The number of addresses in the smallest subblock, which requires 10 addresses, is not a power of 2 either. We allocate 16 addresses. The subnet mask for this subnet can be found as $n_3 = 32 - \log_2 16 = 28$. The first address in this block is 14.24.74.192/28; the last address is 14.24.74.207/28.

If we add all addresses in the previous subblocks, the result is 208 addresses, which means 48 addresses are left in reserve. The first address in this range is 14.24.74.208. The last address is 14.24.74.255.



a. Original block



b. Subblocks

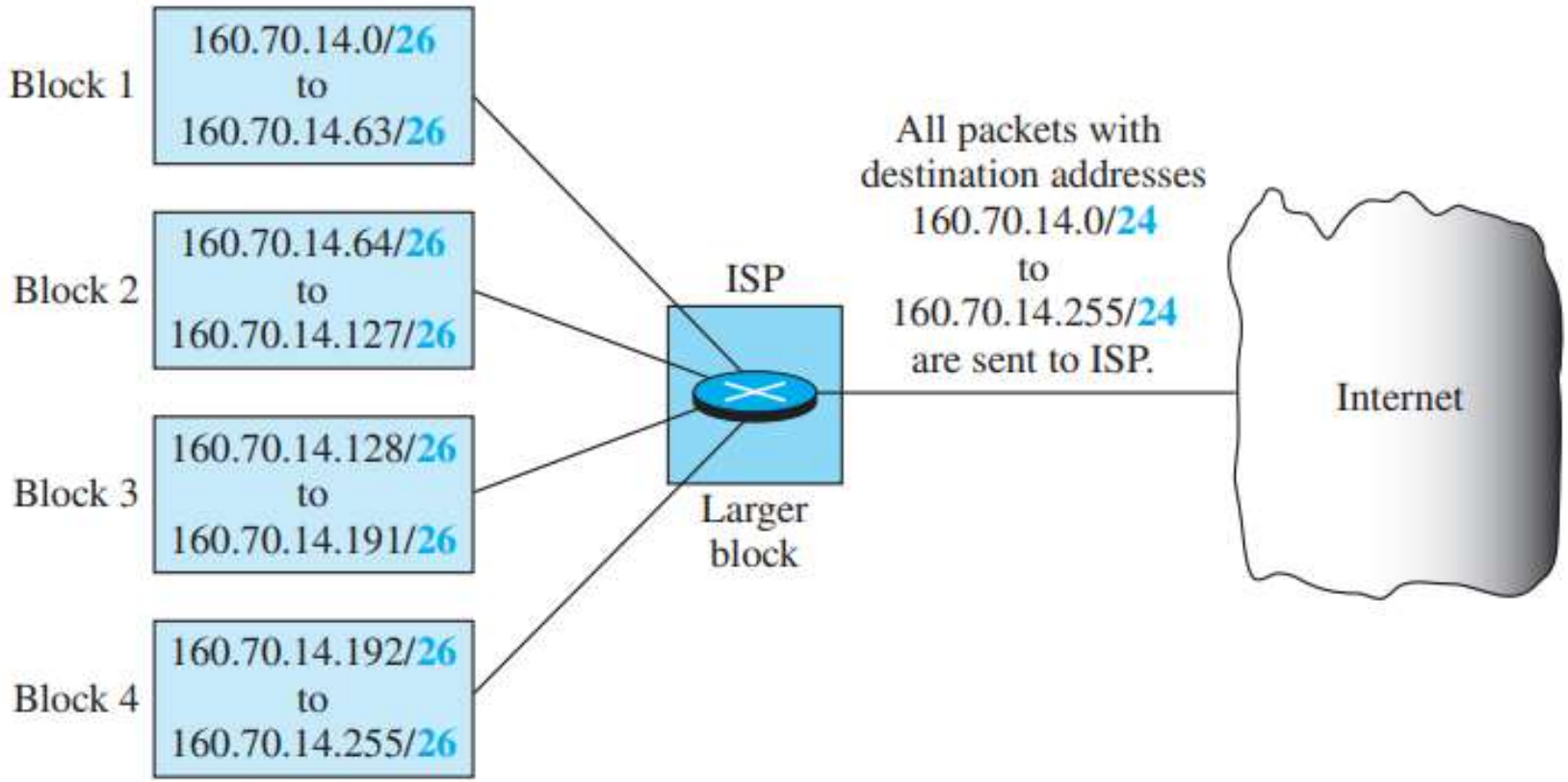
Address Aggregation

- One of the advantages of the CIDR strategy is ***Address Aggregation***.
- Sometimes called ***Address Summarization*** or ***Route Summarization***.
- When blocks of addresses are combined to create a larger block, routing can be done based on the prefix of the larger block.
- ICANN assigns a large block of addresses to an ISP. Each ISP in turn divides its assigned block into smaller sub-blocks and grants the sub-blocks to its customers.

For example:

- Four small blocks of addresses are assigned to four organizations by an ISP.
- The ISP combines these four blocks into one single block and advertises the larger block to the rest of the world.
- Any packet destined for this larger block should be sent to this ISP.
- It is the responsibility of the ISP to forward the packet to the appropriate organization.
- This is similar to the routing process in a postal network.
- All packages coming from outside a country are sent first to the capital and then distributed to the corresponding destination.

Example of address aggregation

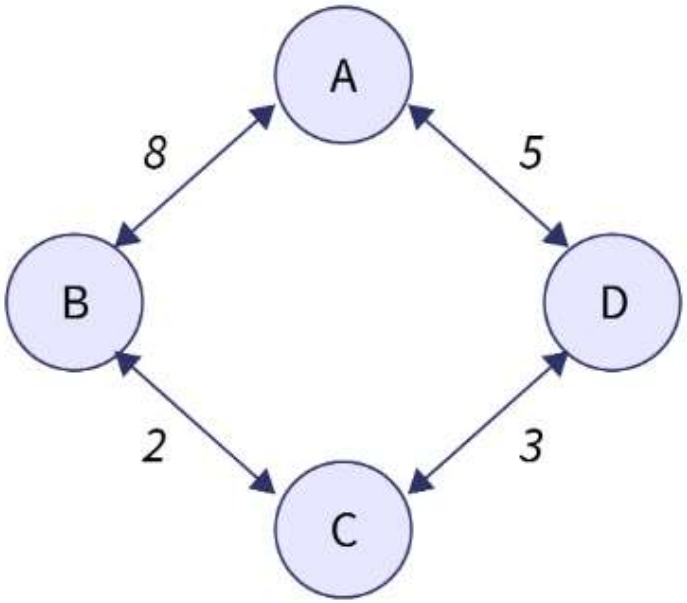


2001:0db8:85a3:0000:0000:8a2e:0370:7334

Distance Vector Routing algorithm

Bellman-Ford's Algorithm

$$D_{xy} = \min\{D_{xy}, (c_{xz} + D_{zy})\}$$



Routing table of A:

Destination	distant	Hop
A	0	A
B	8	B
C	infinity	–
D	5	D

Routing table of B:

Destination	distant	Hop
A	8	A
B	0	B
C	2	C
D	infinity	–

Routing table of C:

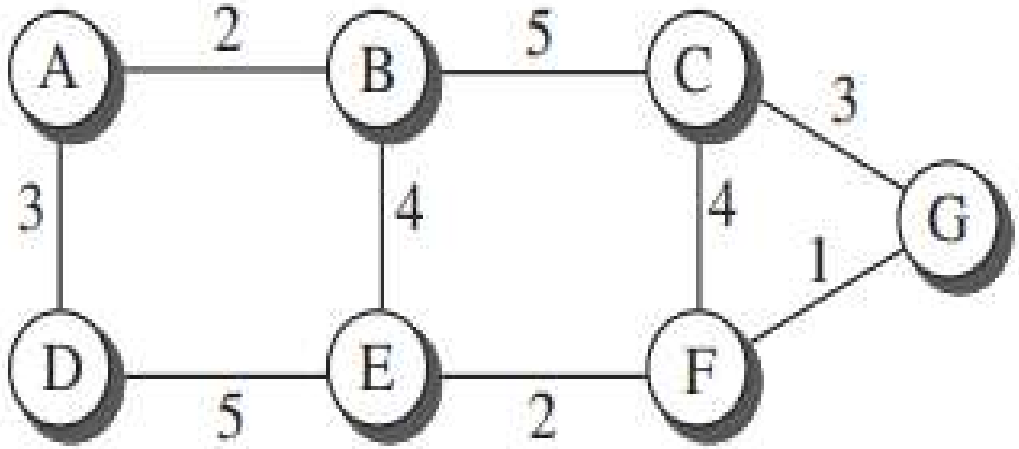
Destination	distant	Hop
A	infinity	–
B	2	B
C	0	C
D	3	D

Routing table of D:

Destination	distant	Hop
A	5	A
B	infinity	–
C	3	C
D	0	D

Link State Routing algorithm

- To create a least-cost tree with this method, each node needs to have a complete map of the network, which means it needs to know the state of each link.
- The collection of states for all links is called the **Link-State DataBase (LSDB)**.
- There is only one LSDB for the whole internet; each node needs to have a duplicate of it to be able to create the ***least-cost tree***.



a. The weighted graph

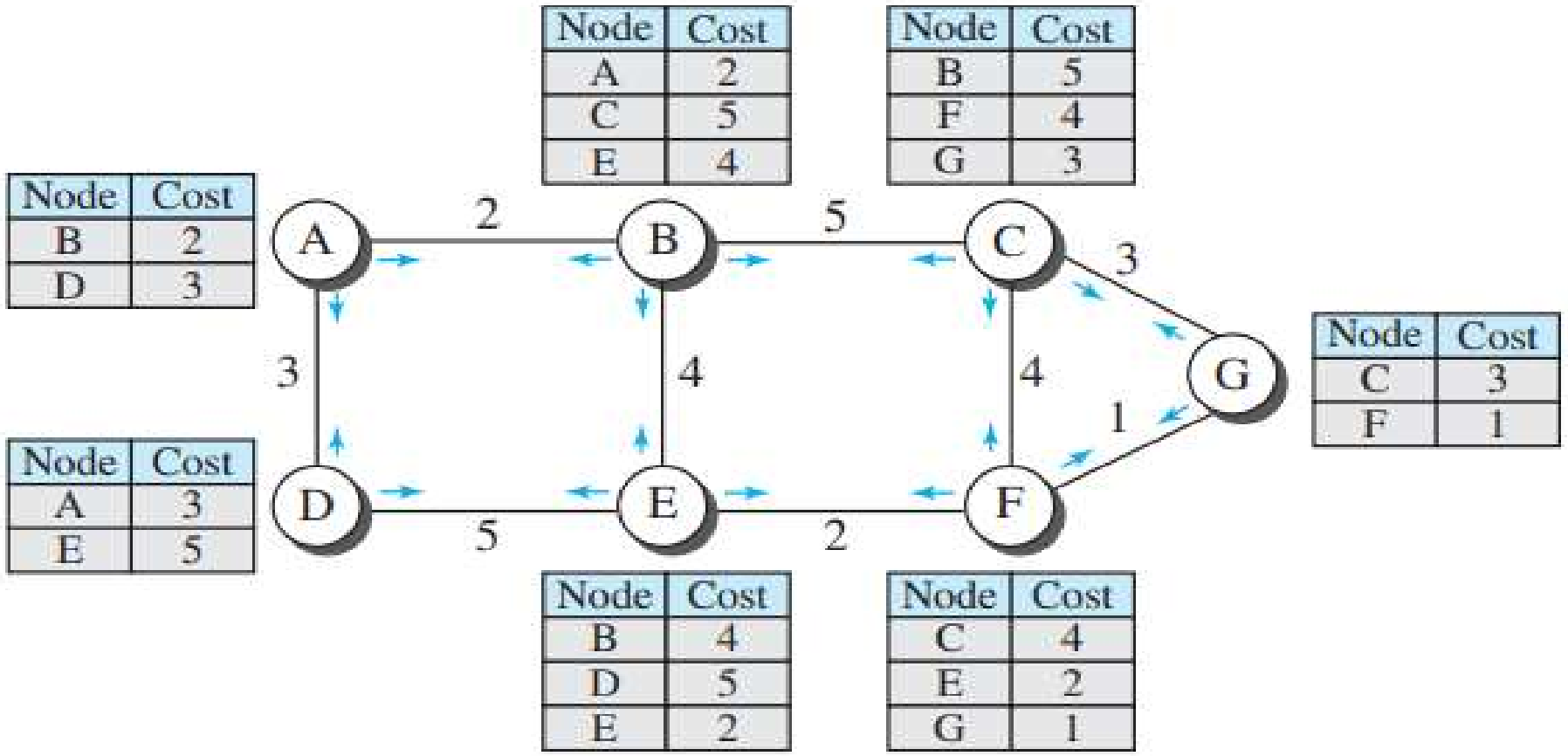
	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

b. Link state database

- Each node can send some greeting messages to all its immediate neighbors (those nodes to which it is connected directly) to collect ***two*** pieces of information for each neighboring node:
    - ✓ The identity of the node
    - ✓ The cost of the link
- The combination of these two pieces of information is called the ***LS packet (LSP)***; the LSP is sent out of each interface.
- The node also have the **Sequence number** and **TTL value** along with the neighbor information.
- This LS Information is shared through **Flooding** with other nodes in the network.

- When a node receives an LSP from one of its interfaces, it compares the LSP with the copy it may already have.
- If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP.
- If it is newer or the first one received, the node discards the old LSP (if there is one) and keeps the received one.
- It then sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the network (where a node has only one interface).
- This **LSDB** is the same for each node and shows the whole map of the internet.
- In other words, a node can make the whole map if it needs to, using this LSDB.

LSPs created and sent out by each node to build the LSDB



Formation of Least-Cost Trees

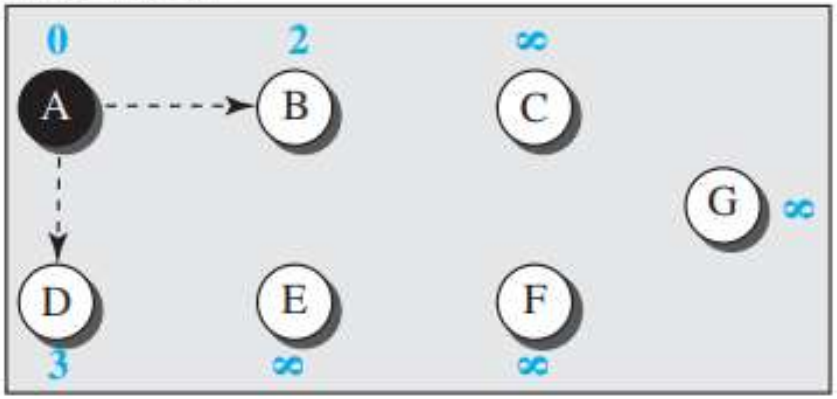
To create a least-cost tree for itself, using the shared LSDB, each node needs to run the famous **Dijkstra's algorithm**. This iterative algorithm uses the following steps:

1. The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.
2. The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
3. The node repeats step 2 until all nodes are added to the tree.

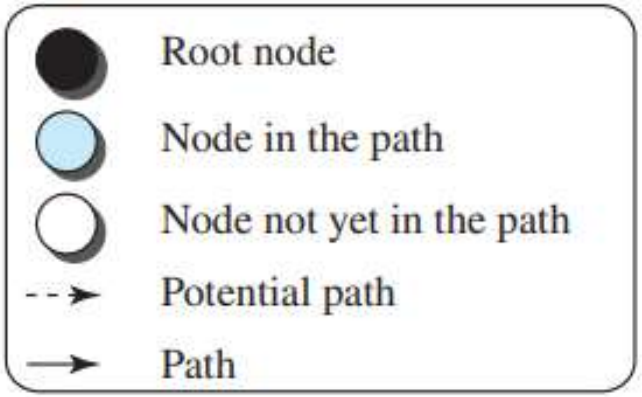
```
1   Dijkstra's Algorithm ( )
2   {
3      // Initialization
4      Tree = {root}          // Tree is made only of the root
5      for (y = 1 to N)       // N is the number of nodes
6      {
7          if (y is the root)
8              D [y] = 0      // D [y] is shortest distance from root to node y
9          else if (y is a neighbor)
10             D [y] = c[root][y]      // c [x] [y] is cost between nodes x and y in LSDB
11         else
12             D [y] = ∞
13     }
14     // Calculation
15     repeat
16     {
17         find a node w, with D [w ] minimum among all nodes not in the Tree
18         Tree = Tree ∪ {w}      // Add w to tree
19         // Update distances for all neighbor of w
20         for (every node x, which is neighbor of w and not in the Tree)
21         {
22             D[x] = min{D[x], (D[w] + c[w][x])}
23         }
24     } until (all nodes included in the Tree)
25  }
```

Least-cost tree

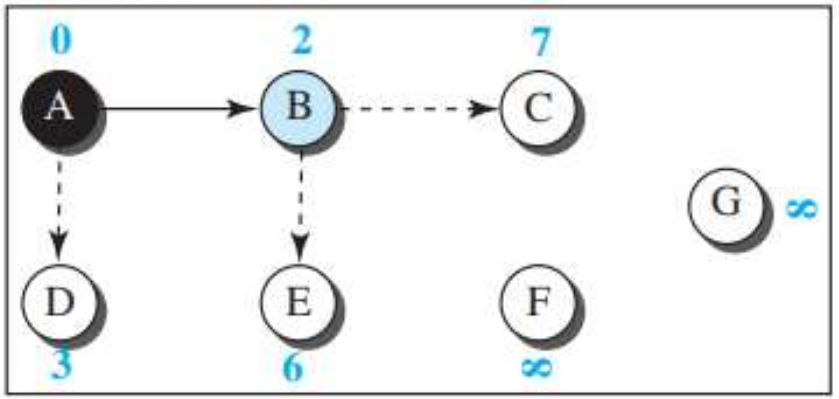
Initialization



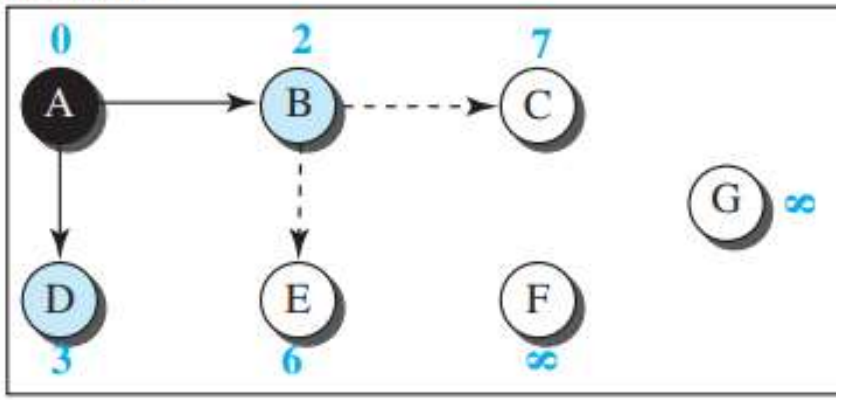
Legend



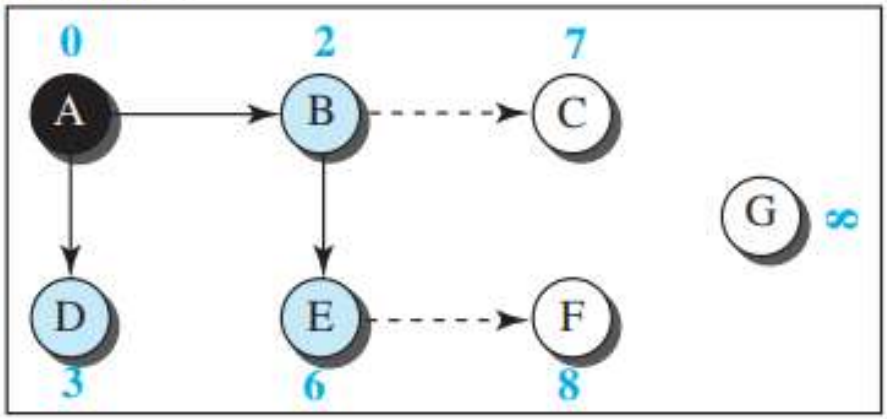
Iteration 1



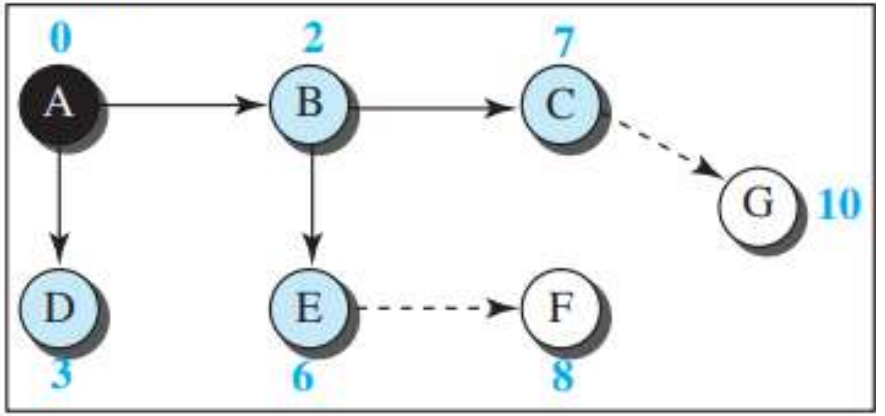
Iteration 2



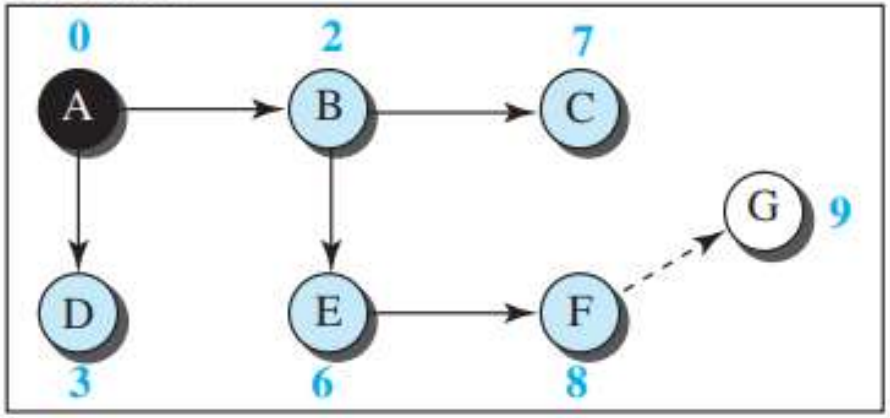
Iteration 3



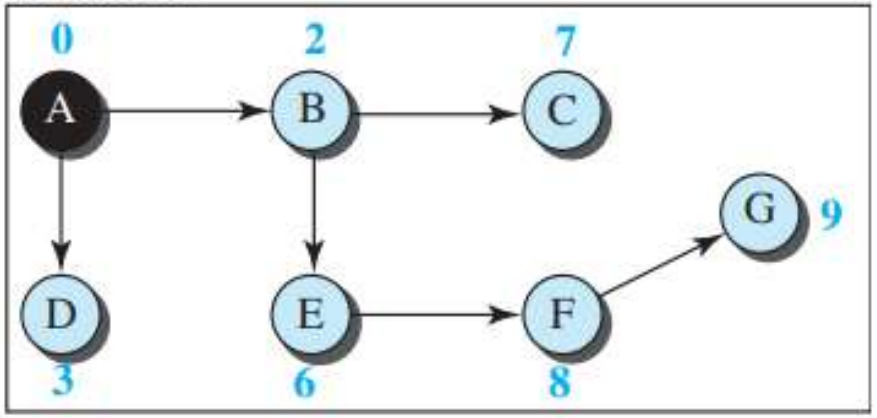
Iteration 4



Iteration 5

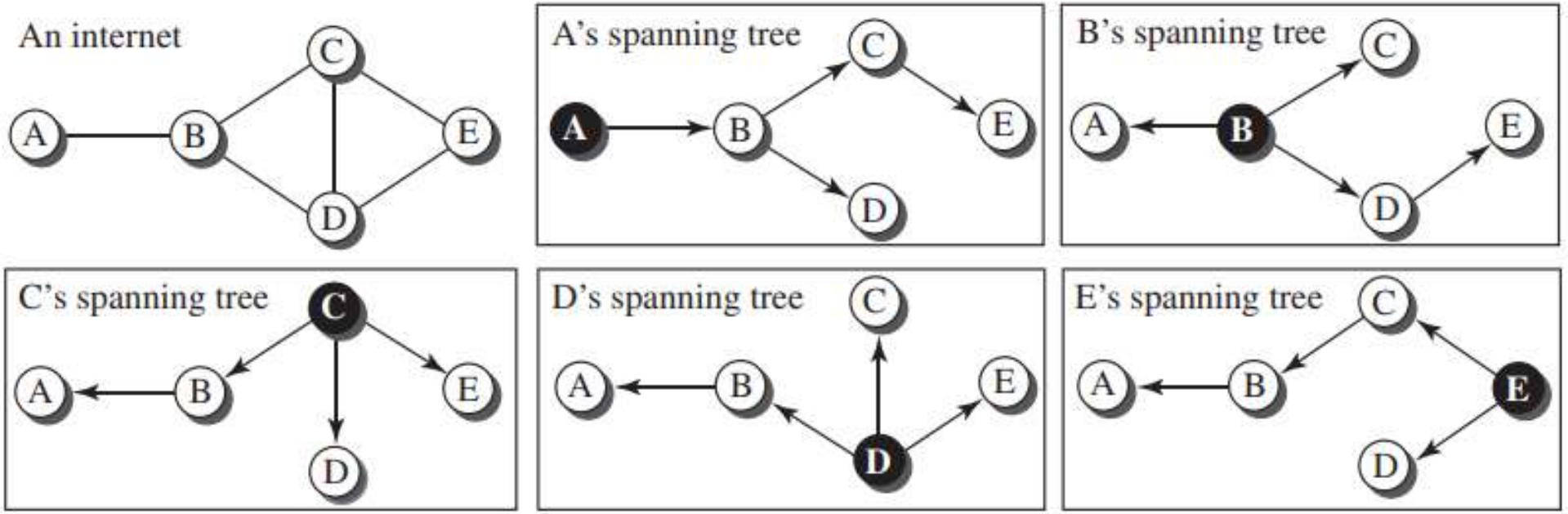


Iteration 6



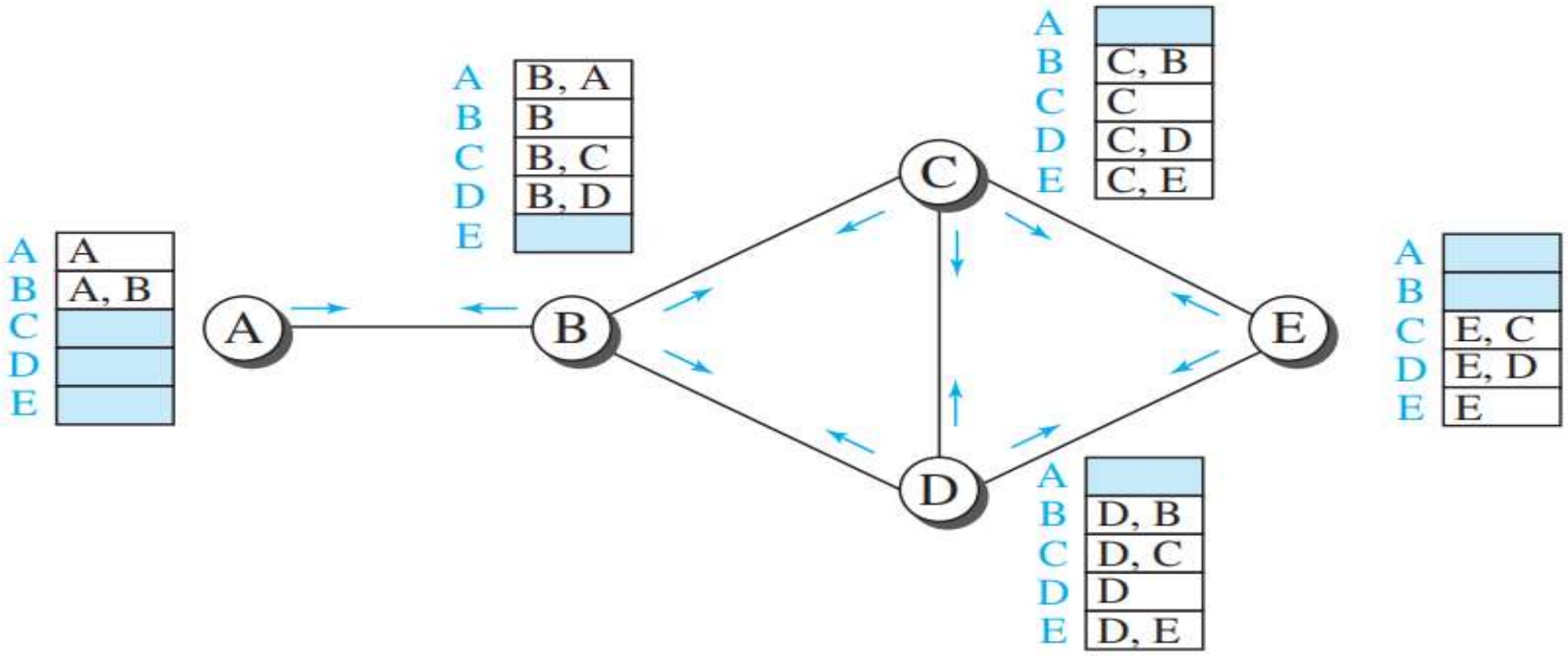
Path-Vector Routing

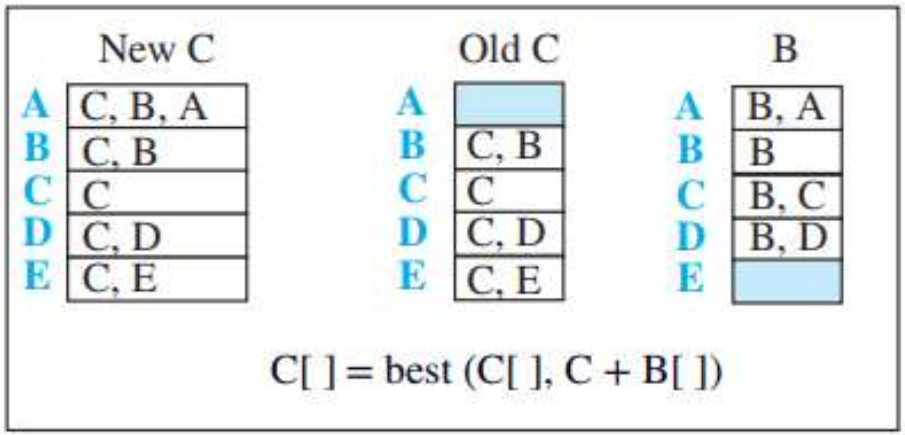
- In path-vector routing, the path from a source to all destinations is also determined by the best spanning tree.



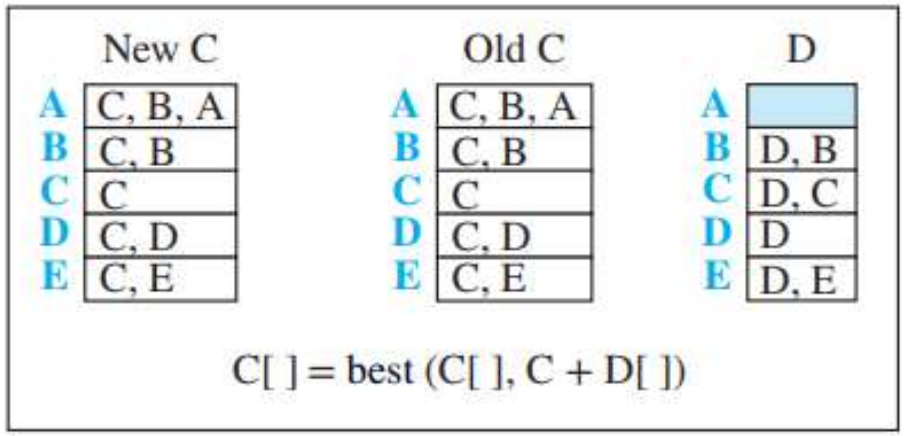
$$\text{Path}(x, y) = \text{best}\{\text{Path}(x, y), [(x + \text{Path}(v, y)]\} \quad \text{for all } v\text{'s in the internet}$$

Path vectors at Initial stage:





Event 1: C receives a copy of B's vector



Event 2: C receives a copy of D's vector

Note:
X [ ]: vector X
Y: node Y

```
1   Path_Vector_Routing ( )
2   {
3     // Initialization
4     for (y = 1 to N)
5     {
6         if (y is myself)
7             Path[y] = myself
8         else if (y is a neighbor)
9             Path[y] = myself + neighbor node
10        else
11            Path[y] = empty
12    }
13    Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14    // Update
15    repeat (forever)
16    {
17        wait (for a vector Path_w from a neighbor w)
18        for (y = 1 to N)
```

```
19          {
20              if (Path_w includes myself)
21                  discard the path                        // Avoid any loop
22              else
23                  Path[y] = best {Path[y], (myself + Path_w[y])}
24          }
25          If (there is a change in the vector)
26              Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27      }
28  } // End of Path Vector
```